Redux Toolkit: The Modern Way

4.3 – Redux Toolkit: The Modern Way

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is Redux Toolkit and why should you use it?"	🟡🔴	★★★★★
"What is createSlice?"	🟡🔴	★★★★☆
"What is configureStore?"	🟡	★★★☆☆
"What is createAsyncThunk?"	🔴	★★★★☆
"How does RTK handle immutability?"	🔴	★★★☆☆

Why Redux Toolkit Exists

Classic Redux requires too much boilerplate. For every feature, you write action types, action creators, reducers, and immutable update logic. Redux Toolkit (RTK) was created by the Redux team to be the official, recommended way to write Redux. It eliminates the boilerplate while keeping all the power.

If someone says "Redux" in 2025, they mean Redux Toolkit. Writing classic Redux from scratch is for learning purposes only.

---

configureStore: Setting Up the Store

`configureStore` replaces `createStore`. It automatically sets up Redux DevTools, includes redux-thunk middleware, and catches common mistakes in development.

JSX Copy

```jsx
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './features/counter/counterSlice';
import todosReducer from './features/todos/todosSlice';

const store = configureStore({
  reducer: {
    counter: counterReducer,
    todos: todosReducer,
  },
});

export default store;
```

That's it. No `combineReducers`, no manual DevTools setup, no middleware configuration. RTK handles all of it.

---

createSlice: Actions + Reducer in One

`createSlice` is the star of Redux Toolkit. It generates action creators and action types automatically from the reducer functions you define.

```
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => {
      state.value += 1;  // You CAN mutate here. Immer handles immutabilit
    },
    decrement: (state) => {
      state.value -= 1;
    },
    incrementByAmount: (state, action) => {
      state.value += action.payload;
    },
    reset: (state) => {
      state.value = 0;
    },
  },
});

// Auto-generated action creators
export const { increment, decrement, incrementByAmount, reset } = counterS

// Reducer to plug into the store
export default counterSlice.reducer;
```

Notice the "mutation" syntax: `state.value += 1`. In classic Redux, this would be a bug. In RTK, it's safe because RTK uses **Immer** under the hood. Immer intercepts your mutations and produces a proper immutable update behind the scenes. You write mutating code, but the result is still immutable.

Using it in a component

JSX Copy

```
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, reset } from './counterSlice';
```

```
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() => dispatch(reset())}>Reset</button>
    </div>
  );
}
```

createAsyncThunk: Handling Async Operations

Real apps fetch data from APIs. `createAsyncThunk` handles the async lifecycle (pending, fulfilled, rejected) automatically.

JSX Copy

```jsx
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

// Define the async operation
export const fetchUsers = createAsyncThunk('users/fetchUsers', async () =>
  const response = await fetch('/api/users');
  return response.json();
});

const usersSlice = createSlice({
  name: 'users',
  initialState: {
    items: [],
    loading: false,
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
```

```
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false;
        state.items = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      });
  },
});

export default usersSlice.reducer;
```

JSX Copy

```jsx
// Component usage
function UserList() {
  const dispatch = useDispatch();
  const { items, loading, error } = useSelector(state => state.users);

  useEffect(() => {
    dispatch(fetchUsers());
  }, [dispatch]);

  if (loading) return <p>Loading ...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {items.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
}
```

`createAsyncThunk` generates three action types automatically: `users/fetchUsers/pending`, `users/fetchUsers/fulfilled`, and `users/fetchUsers/rejected`. The `extraReducers` builder handles each one.

Classic Redux vs Redux Toolkit

Feature	Classic Redux	Redux Toolkit
Action types	Define string constants manually	Auto-generated from slice name + re
Action creators	Write functions manually	Auto-generated by `createSlice`
Immutable updates	Spread syntax everywhere	Write "mutations" safely with Immer
Store setup	`createStore` + manual middleware	`configureStore` with batteries incl
Async operations	Manual with redux-thunk or redux-saga	`createAsyncThunk` with built-in life
DevTools	Manual setup	Included automatically
Boilerplate	A lot	Minimal

Folder Structure for RTK

```
src/
  app/
    store.js                // configureStore
  features/
    counter/
      counterSlice.js       // createSlice + async thunks
      Counter.jsx           // Component
    todos/
      todosSlice.js
      TodoList.jsx
    users/
      usersSlice.js
      UserList.jsx
```

Each "feature" gets a folder with its slice and component(s). The slice file contains the reducer, actions, and async thunks all in one place.

Common Mistakes

Mistake 1: Mutating state outside of createSlice. Immer only works inside `createSlice` reducers and `createReducer`. If you write `state.value += 1` in a regular function, it's a real mutation.

Mistake 2: Forgetting `extraReducers` for async thunks. `createAsyncThunk` generates actions, but you must handle them in `extraReducers` for them to actually update state.

Mistake 3: Not exporting both the actions and the reducer. Export `slice.actions` for components to dispatch, and `slice.reducer` for the store configuration. Forgetting either one is a common source of "nothing works and there's no error."

Interview Answer

of the boilerplate. `configureStore` sets up the store with DevTools and middleware automatically. `createSlice` generates action creators and action types from your reducer functions, and uses Immer so you can write mutation-like code that produces immutable updates safely. For async operations, `createAsyncThunk` handles the pending/fulfilled/rejected lifecycle. In a modern React project, I would never use classic Redux directly. RTK is simpler, less error-prone, and the Redux team themselves recommend it."

✅ Key Takeaways

- **Redux Toolkit (RTK)** is the official, recommended way to use Redux. Don't write classic Redux manually.
- `configureStore` replaces `createStore` with automatic DevTools, middleware, and error checking.
- `createSlice` auto-generates actions and lets you write "mutations" safely via **Immer**.
- `createAsyncThunk` handles async operations with built-in pending/fulfilled/rejected states.
- Handle async thunk results in `extraReducers`, not `reducers`.
- Organize by **feature folders**: each feature gets a slice file and a component file.

✨ QUICK CHECK — no marks, just for you.

1. You replace createStore with configureStore and pass only a reducer map. According to the lesson, what do you still have to wire up manually to get Redux DevTools working and to dispatch a basic async thunk?

- ○ redux-thunk must be added to a middleware array, but DevTools is automatic

- Nothing — configureStore sets up DevTools and includes redux-thunk middleware automatically
- DevTools must still be composed in manually, but redux-thunk is included

Check answer

2. A slice is created with name 'counter' and a reducers object containing increment, decrement, and reset. What must you additionally write so components can dispatch these updates?

- String action type constants, since createSlice only generates the action creators
- A separate createAction call for each reducer key before they can be dispatched
- Nothing — createSlice generates the action creators and action types, and you export slice.actions
- Action creator functions for each reducer, since createSlice only generates the action types

Check answer

3. Inside a createSlice reducer you write state.value += 1, which would be a bug in classic Redux. Why is the resulting state update still immutable here?

- createSlice deep-clones the state before each reducer runs, so the original object is untouched
- Redux compares the old and new state and re-creates the object only when a change is detected
- Immer intercepts the mutation and produces a proper immutable update behind the scenes

The state argument is frozen, so the assignment is silently ignored and the store keeps the old value

Check answer

Previous
Redux from Scratch: Core Principles

Next
Redux Middleware & Side Effects

LEARN MORE

ARTICLES & DOCS

- Redux Toolkit — official docs
  redux-toolkit.js.org
- Redux Toolkit — Quick Start
  redux.js.org

VIDEOS



- Redux Toolkit — complete tutorial
  YouTube



- Learn Redux Toolkit with a project
  YouTube



- React 18 with Redux Toolkit
  YouTube

Hand-picked references for this lesson.